

2주차 추가 과제

한찬희

- Q.1 과 Q.4의 베릴로그 RTL에서 그래픽 장치를 다루는 방식의 차이점에 대하여 설명할 것

Q.1에서는 640x480 VGA 환경으로 동작하고 색표현은 RGB각 1비트씩 8색으로 제어한다.

Q.4에서는 128x64 GLCD환경을 기준으로 동작하고 색표현은 1비트 흑백으로 표현한다.

Q.1에서는 vga_x와 vga_y가 cannon_x, bullet_x y, invader_x y 의 영역 범위에 포함되는지 조합논리 비교연산자로 실시간 계산한다.

Q.4에서는 lcd_x y 와 lcd_data를 직접 제어한다.

- Q.10 의 실습을 수행 할 것. 스프라이트 시현 방식에 대하여 아두이노 C++와 베릴로그 RTL의 차이를 비교 설명 할 것.

Q.10을 실행하면 흑백 체크무늬가 나온다. 스페이스바를 누르면 실행이된다. 외계인이 4X4로 나오고 스페이스바를 눌러 외계인을 공격한다. 공격하면 외계인이 없어진다. 반대로 우주선이 외계인의 공격을 받으면 오른쪽위의 목숨이 사라지며 부활한다. 죽고 부활하는데 오래걸려 오류가 생긴줄 알았다. 외계인의 공격을 피하고 외계인을 맞추기 위해 방향키를 누르면 움직이지 않는다. 꺾 누르면 그제서야 맨왼쪽 혹은 맨 오른쪽으로 움직이게 된다. 외계인의 공격이 있으면 이동시 무조건 죽게 된다.

아두이노 c++은 CPU가 명령어를 한줄씩 실행한다. RAM에 전체 그림을 다 그린뒤 전송하거나 화면에 점을 그리는 동안 CPU가 대기하고 있어야한다. 화면에 표출할 스프라트가 늘어나면 CPU점유율이 늘어나고 프레임 레이트가 떨어진다. CPU가 Bus를 통해 Flash Memory 주소를 읽어오는 i/o병목 및 메모리 접근 지연이 발생한다.

베릴로그 RTL방식은 전용 카운터가 클럭에 맞춰 주소를 1씩올릴 때 연산이 한 클럭 주기에 병렬적으로 이루어진다. 스프라이트 개수가 늘어나도 MUX와 비교기 회로만 늘어나고 클럭 주파수 고정속도로 화면을 완벽하게 실시간으로 출력한다. 주소선과 데이터선이 하드웨어 와이어로 직접 연결되어 지연 시간 없이 픽셀 비트 패턴을 디코딩한다.

- Q.11 의 실습을 수행 할 것. 좌우 화살표 키보드 입력에 대하여 이상 반응을 보이는 이유를 찾을 것.

Q.11을 실행하니 오류가 생겼다. 확인해보니 systemc를 찾는 경로가 잘못되어 경로를 수

정해주니 작동하였다. 좌우 코드를 수정하니 외계인이 움직이는 속도처럼 움직이게 되었다. 이상하게 작동하는 이유는 하드웨어 클럭만큼 그대로 키보드 입력이 되었기 때문이다. 클럭이 50MHz로 작동하는데 입력도 좌표에서 x-1을 하는 동작도 클럭만큼 입력되기 때문이다. 그래서 우주선이 순식간에 벽에 닿게 된다

Q.11의 SystemC 테스트 벤치가 변경된 점을 찾아볼 것.

while (true) {wait(16.6, SC_MS)구문을 사용하여 while 무한이나 Sleep함수를 안쓰고 16.6ms(60hz)를 구현하였다.

SDL_PumpEvents();

Const Uint8 staate = SDL_GetKeyboardsState(NULL); 이문구를 통해 키보드를 눌러도 입력을 제때 인식 못하는 현상을 차단하였다.

If(state != nullptr) 을 통해 키보드 상태배열을 받아온 후 포인터가 비었는지 검사한다. 이를 통해 메모리 참조 오류로 인해 시뮬레이터가 강제 종료되는 것을 막는다.

btn_left.write(state[SDL_SCANCODE_LEFT] ? true : false);

state 배열에서 읽어온 값은 8비트 정수 타입이고 SystemC 하드웨어 포트(1비트)에 넣기 위해 삼항연산자를 사용하였다. 값이 1과 0으로 엄격하게 논리 변환을 수행한다.

-자신만의 문장으로 이 둘(소프트웨어 언어와 하드웨어 언어)을 비교할 수 있도록 정리 하시기 바랍니다.

소프트웨어는 CPU가 메모리 주소를 따라 명령어를 순차적으 실행한다. 반면 하드웨어는 연산을 순서를 나열한 것이 아닌 물리적으로 회로를 배치하고 연결하는 작업이다.

하드웨어 언어로 작성했다고 꼭 회로로 합성되는 것은 아니다. 회로로 합성되려면 레지스터 사이에 데이터가 어떻게 전송되고 조합논리 회로를 거쳐지는지에 대해 RTL레벨로 기술되어야 한다.

a. 소프트웨어 언어에서는 int나 char 처럼 자료형을 사용한다. 하드웨어에서는 [LSB:RSB] 처럼 비트 단위로 핀과 버스를 제어해야한다.

b. 실제 회로에서는 1(true), 0(false) 뿐만 아니라 X(unknown), Z(high-impedence)가 존재하기 때문에 이 4-state 논리를 표현하고 연산할 수 있는 데이터 타입이 있어야한다.

c. 많은 게이트와 플립플롭이 클럭에 맞춰 병렬적으로 동작한다. 이를 소프트웨어 환경에서 동작시키게 하려면 SystemC의 wait(10, CS_N)처럼 시간관리를 해야한다.

d. 하드웨어 회로는 신호는 신호의 상승이나 하강 에지 같은 특정 사건이 발생하는 순간

에 맞춰 동작한다. C++객체 안에서 사건을 감지하고 프로세스를 깨우는 매커니즘 (sensitive << clk.pos())이 있어야한다.